The Devil's Gambit

1. Project Overview

The Devil's Gambit is a single player roguelite card game where you play against demons in a game of blind card picking to win your soul back. Both players share a grid of hidden cards, manipulating it in real time using special cards to gain the upper hand. Defeat each demon to grow stronger, or lose and face the consequences.

2. Project Review

While designing the game, I got heavily inspired by the card game Inscryption and the board game Connect 4. Inscryption's design is very simple. You play cards to damage the enemy, gain points to tip over the scale and win. Despite the simplicity, the game adds modifiers to let players "rig" the game and plan things out in hard situations. I figured what if I use the Inscryption style game and try to fuse it with another seemingly simple game: Connect 4? Connect 4 is straightforward; who gets 4 in a row wins. This leads me to a game that borrows Connect 4's grid and row system, and the scale-based scoring and helper card system from Inscryption.

Unlike Inscryption, where the board is managed by a single player, The Devil's Gambit introduces a shared contested grid where both players actively manipulate the same space in real time. The helper card system creates a direct strategic exchange rather than a one-sided game of resource management.

3. Programming Development

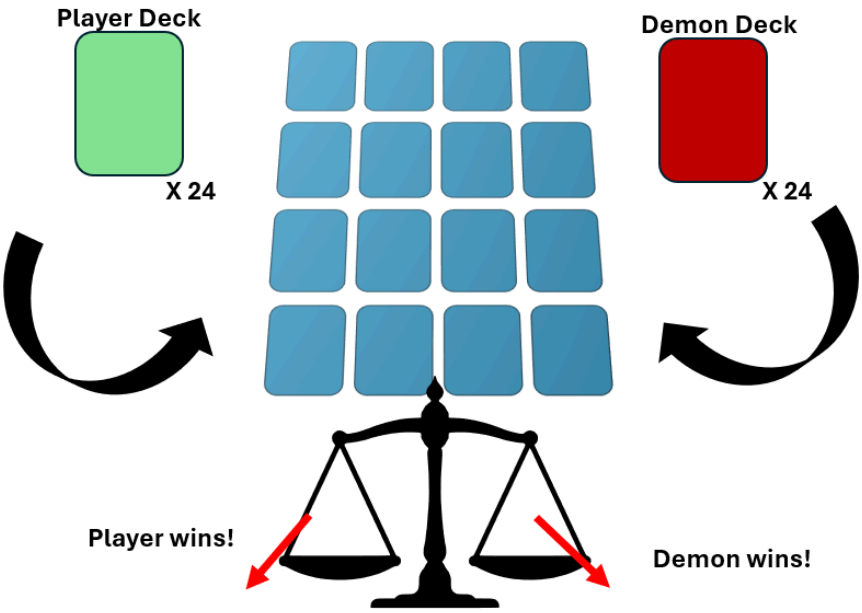
3.1 Game Concept

You're sitting on a table, where a demon you've "accidentally" summoned sits across from you. The demon wants your soul, but you disagree. To solve this, they invite you to play a game of blind card picking by their own rules. The only way out to freedom is to outsmart the demon at their own game.

The game is simple. You must out-score the opponent by a specific margin. The score is represented by a scale, where both score points by picking cards on the table. First to tip the scale past the threshold wins the game.

On the table, there's a 4x4 grid of cards that are laid face down. Each card is drawn from both players' decks. You can't see the values until you flip them. Both players' decks start with 24 cards.

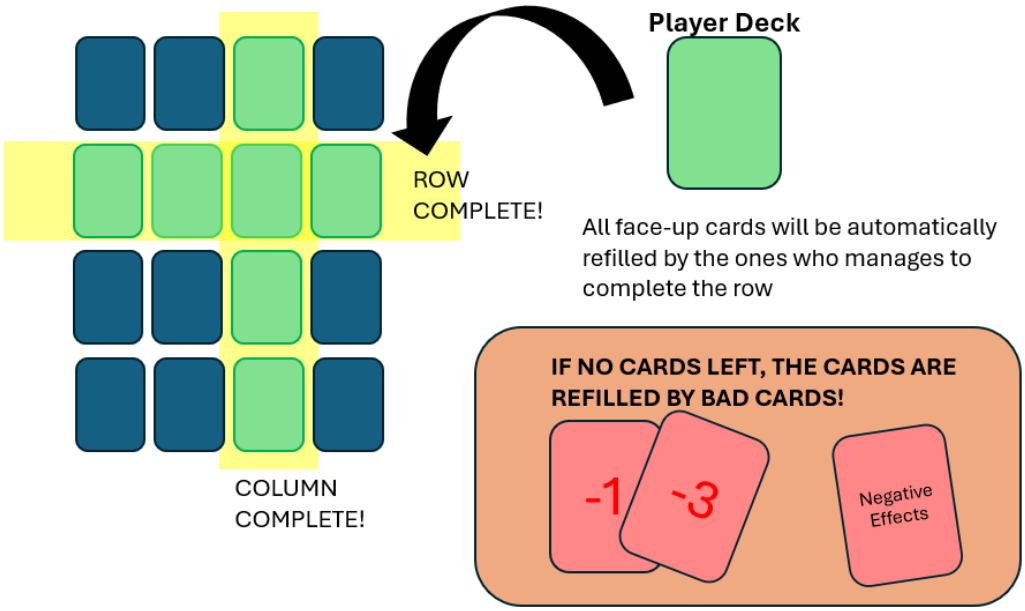
Goal: Outscore the demon past the threshold.



When players pick a card, the player gets points from it. The card stays face up and visible on the grid. However, when a row or column of cards in the grid is fully flipped, it refills with cards from the player that completes it. This gives the player an advantage to have better scored cards on the grid that they have upgraded earlier in the game.

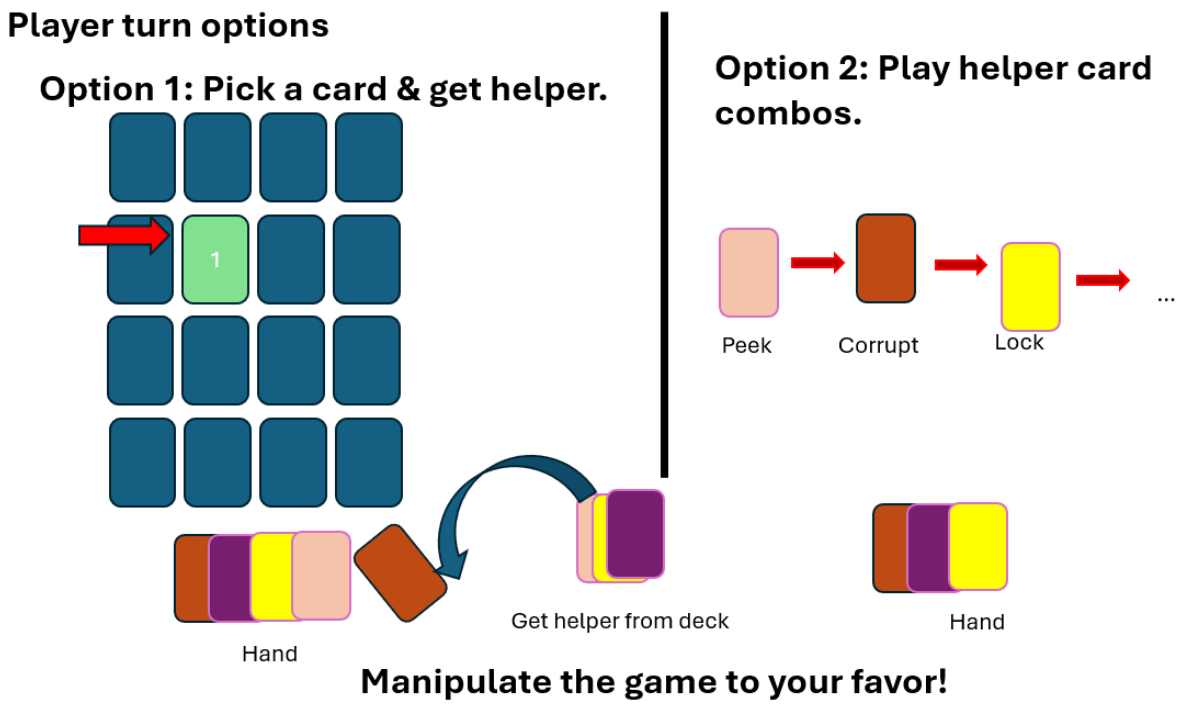
If the player has no cards left, something worse may take its place. The cards may have a negative score or side effects when picked.

Grid Refilling



Each player takes turns to decide to pick a card from the grid and receive a helper card, or play combos of helper cards from their hand.

Helper cards can be used to manipulate or "rig" the game to the player's favor. For example, a player can use Peek to see the value of cards before picking, Corrupt to make a card lower in value, Lock to make the opponent unable to pick it, or Trap to drain the points when they pick the selected cards. It can create devastating combos to win the game.



You get to meet new demons each session. Your foe, the demon, has a unique characteristic for each of them. They pick cards each turn based on their own agenda that the player may be able to figure out. They can also periodically fire passive effects that disrupt your plans. The game features three demons of increasing power, each with their own unique behavior and passive effects.

When you successfully defeat a demon, you get to choose a drop to upgrade your cards, or get new Helper cards. Then, you face the next one. If you lose, you restart the game.

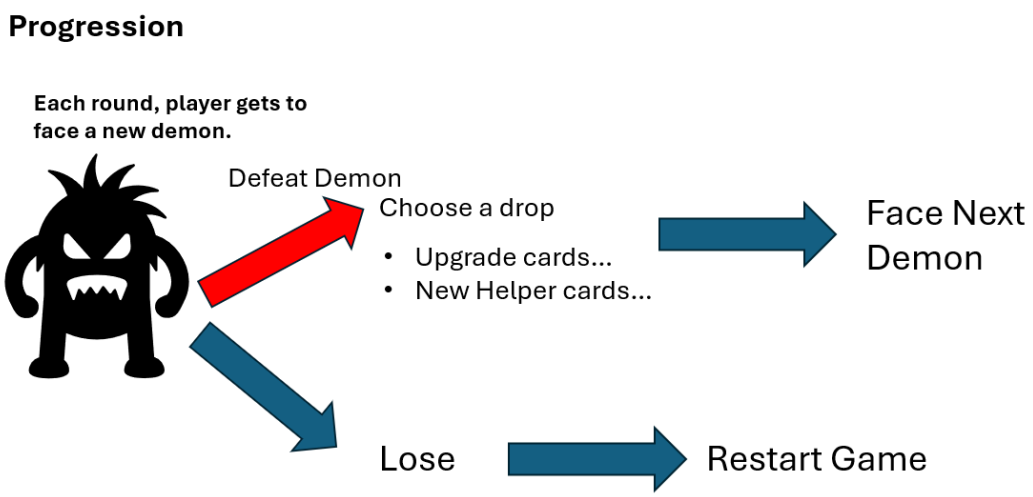


Figure out how to beat each demon, upgrade, and progress.

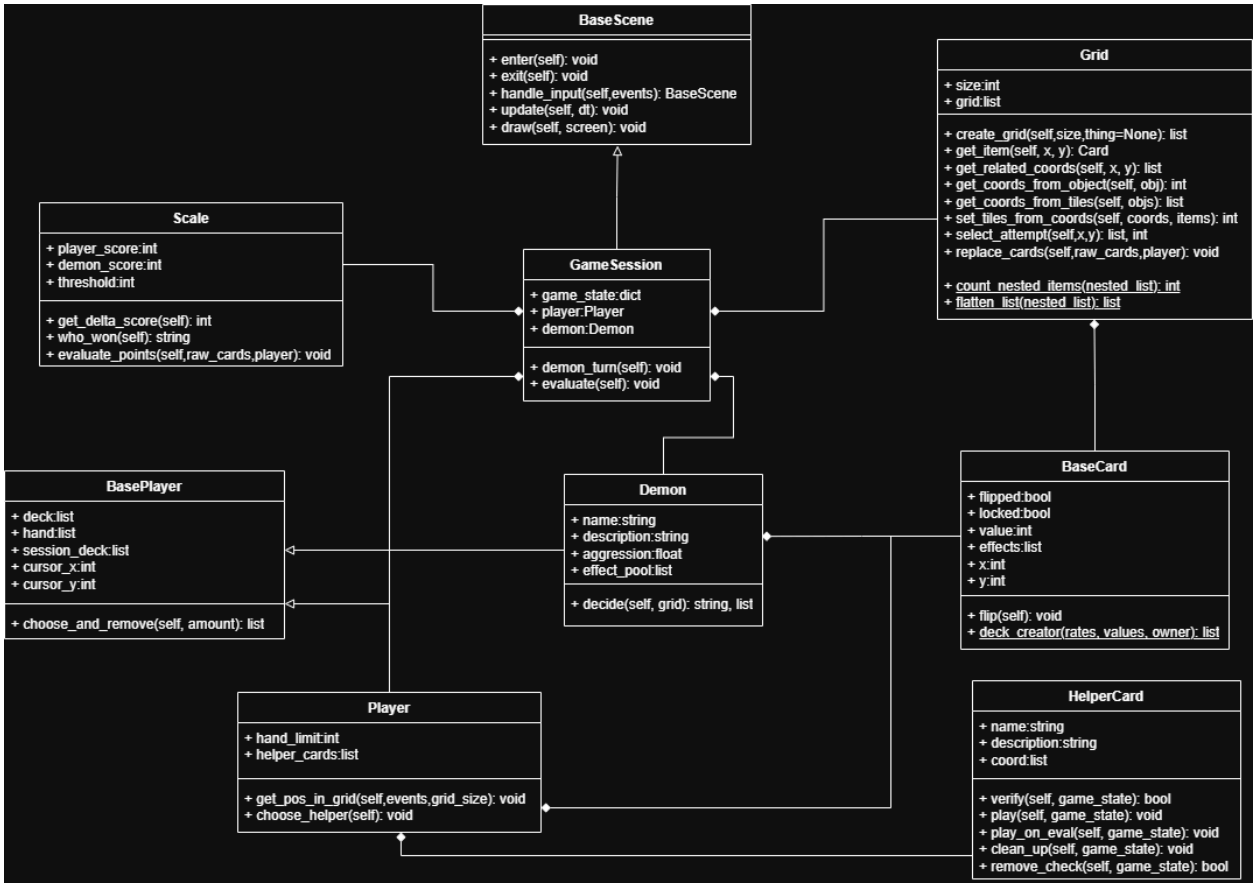
3.2 Object-Oriented Programming Implementation

Class	Role	Key Attributes and Methods
BaseScene	Serves as the base scene where it represents a part of a game (eg. cutscene, main menu, or game round.) It provides a framework for switching from one scene to another.	enter() exit() handle_input() update() draw()
GameSession	Inherits from BaseScene. Manages the main game loop, turn order, win/lose conditions, and scene transitions.	game_state:dict player:Player demon:Demon demon_turn() evaluate()
Grid	Stores and manages a 4x4 card grid. Handles card flipping, row/column completion detection, and refill logic.	size:int grid:list create_grid() get_item() get_related_coords() get_coords_from_object() get_coords_from_tiles()

		set_tiles_from_coords() select_attempt() replace_cards() count_nested_items() flatten_list()
Scale	Tracks both players' scores. Calculate the score difference, keep track with the current session's score, and determine when the threshold is reached.	player_score:int demon_score:int threshold:int get_delta_score() who_won() evaluate_points()
BasePlayer	Manages the deck, and hands for each game session.	deck:list hand:list session_deck:list cursor_x:int cursor_y:int choose_and_remove()
Player	Inherits from BasePlayer. Manages the player's deck, helper card, hand, and processes player input each turn.	hand_limit:int helper_cards:list get_pos_in_grid() choose_helper()
Demon	Inherits from BasePlayer. Uses weighted RNG and aggression values to decide between picking cards and firing passive effects.	name:string description:string aggression:float effect_pool:list decide()
BaseCard	Represents a grid card. Stores value, owner, potential effects and face up/down state.	flipped:bool locked:bool value:int effects:list x:int y:int flip()

HelperCard	Base class for all helper cards. Contains a method to override by each subclass to execute its effect.	name:string description:string verify() play() play_on_eval() clean_up() remove_check()

UML Diagram



3.3 Algorithms Involved

- ***Weighted RNG for demon choice:*** *Each turn the enemy must choose to either pick a card from a grid, or fire a passive effect. This decision is made using weighted RNG, where each action is assigned a based weight (eg. pick a card 70% and fire a passive 30%) The weights change based on the game state. For instance, if the player*

is winning by a margin, increase the weight toward firing a passive effect. This makes the demon to be more dynamic and responds to the player's choices rather than purely random.

The demons have a unique set aggression value that adjusts the base weights differently. A low aggression demon may prioritize card picking to build control, while a high aggression demon may shift weight heavily towards firing passive effects to disrupt the player more frequently. This gives each demon a distinct playstyle that a player can learn to read over time.

- ***Rule based logic for demon behavior:*** *Once the demon decides to pick a card, rule based logic determines what it selects. Each demon has their own set priorities to follow. For example, a demon may: Prefer cards that would complete a row the demon controls, avoid cards that are locked by the player, or cards that have a known high value.*

- ***Rule based logic for helper card effects:*** *Design algorithms that make cards obey the current game state. For instance, Peek should only show cards that are faced down, or Lock prevents the opponent from selecting a specific card for a turn.*

- ***Command pattern for helper card execution:*** *Helps for executing effects from the played card that changes the game.*

- ***Row/column completion:*** *Check if the grid has valid rows and columns and executes a method afterwards.*

- ***State Machine:*** *Helps the game to ease from one scene to another, and player rounds.*

4. Statistical Data (Prop Stats)

4.1 Data Features

Feature	Why is it good to have this data? What can it be used for	How will you obtain 100 values of this feature data?	Which variable and which class will you collect this from?	How will you display this feature data (via summarization statistics or via graph)?
Points gained per turn	Shows the player playstyle. Is the player playing aggressively or cautiously?	Record every card pick across multiple sessions. Each pick generates a value.	Points gained from each turn from Scale class. Eg. player_score	Summarization statistics (mean, SD, min, max)
Board value per turn	Shows how the grid's total value shifts over time. May determine if the player prefers upgrading the card value or helpers.	Get a snapshot of the grid every turn.	The sum of all points from Card class, given in Grid class. Eg. board_value and value	Histogram
Helper card usage frequency	Identify which helper cards players rely on the most. Useful for balancing the game.	Log every helper card played in each round	Name of the card that has been played from HelperCard	Bar Chart
Board card count (Player vs Demon)	Shows who is winning the board control by tracking how many cards each side owns on the grid.	Record card ownership count for each turn before both players run out.	Owner attribute from Card class. Accessing via Grid class	Stacked Bar Graph

Point difference per turn	Shows momentum of the game. Who is winning and when, and how chaotic the scale swings.	Calculate delta between player and demon every turn.	Delta from Scale class' get_delta_score( ) method.	Summarization statistics (mean, SD, min, max)

4.2 Data Recording Method

The game will store the data in a CSV file, which will be separated for each statistic.

4.3 Data Analysis Report

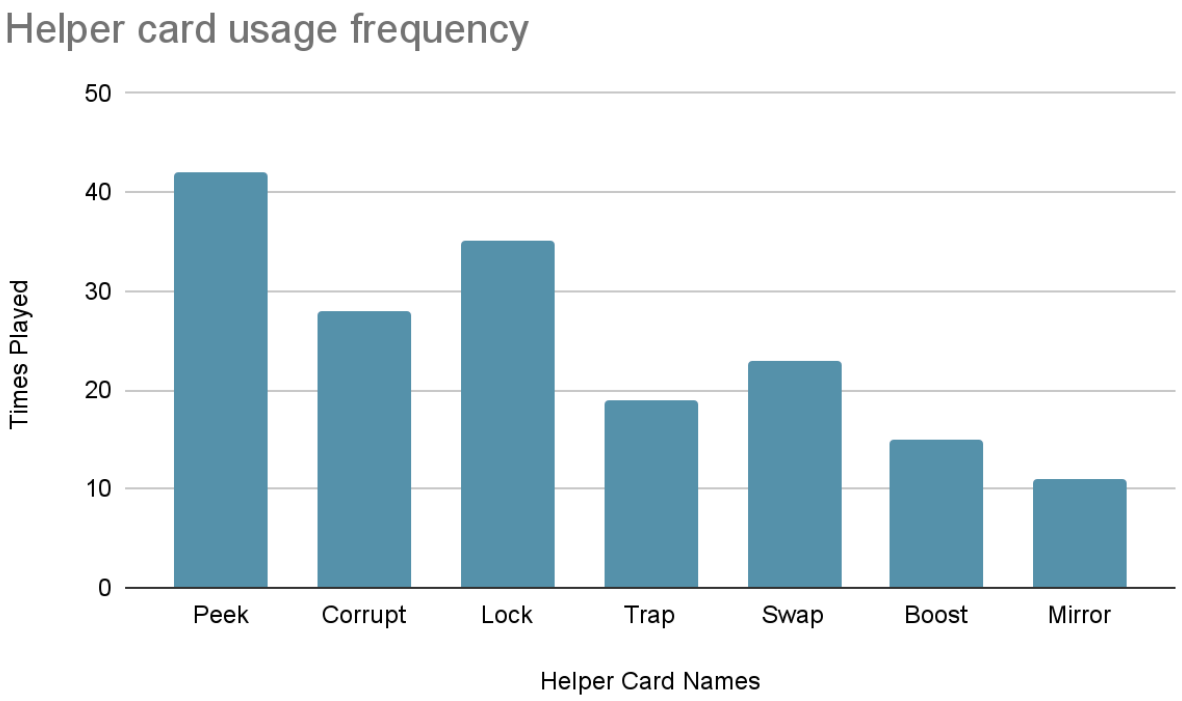
For the analysis, it will use mean, standard deviation, and trend analysis across sessions. Summarized statistics will be presented in a table, followed by the following graphs:

	Feature Name	Graph Objective	Graph Type	X-Axis	Y-Axis
Table	Points gained per turn and Point difference per turn	Show mean, SD, min and max of both features to summarize overall player performance and game momentum.	Table	-	-
Graph 1	Helper card usage frequency	Compares how often each helper card type is played across a session.	Bar Chart	Helper Card names	Times played
Graph 2	Board card count (Player vs Demon)	Shows how board control shifts between player and demon over time.	Stacked Bar Graph	Turn number	Number of cards on board.

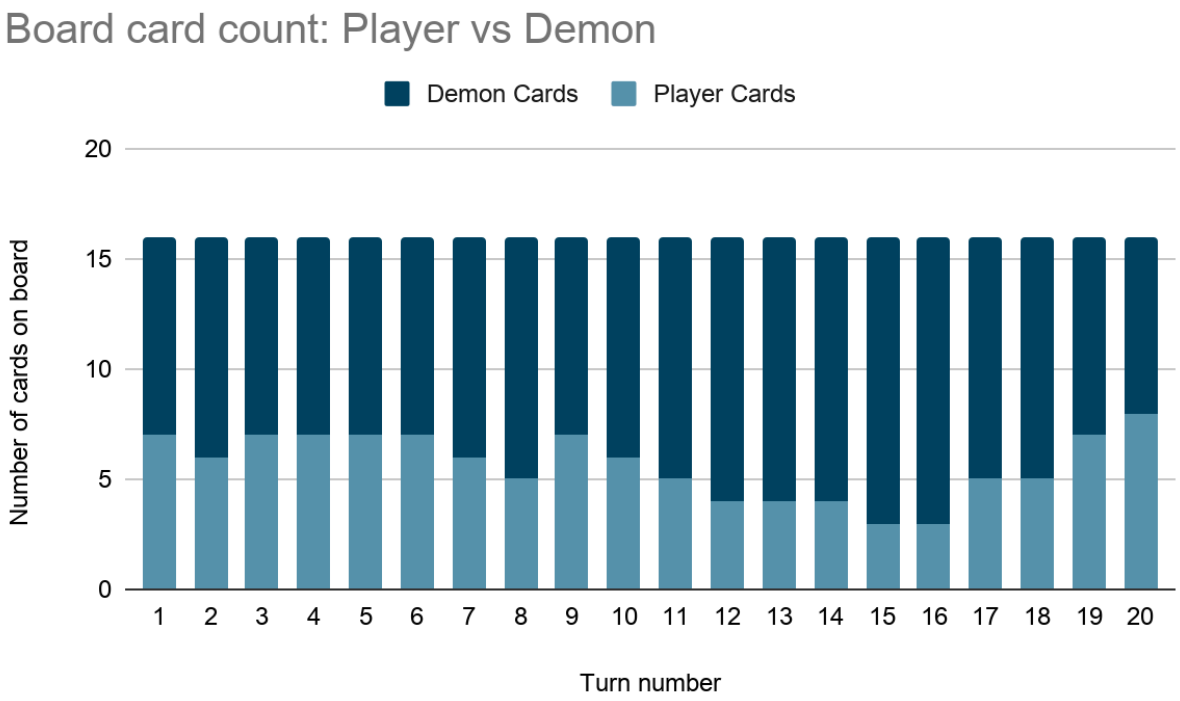
Graph 3	Board value per turn	Shows what value ranges the board tends to sit at most frequently.	Histogram	Board value range.	Frequency (number of turns)

Expected Graphs

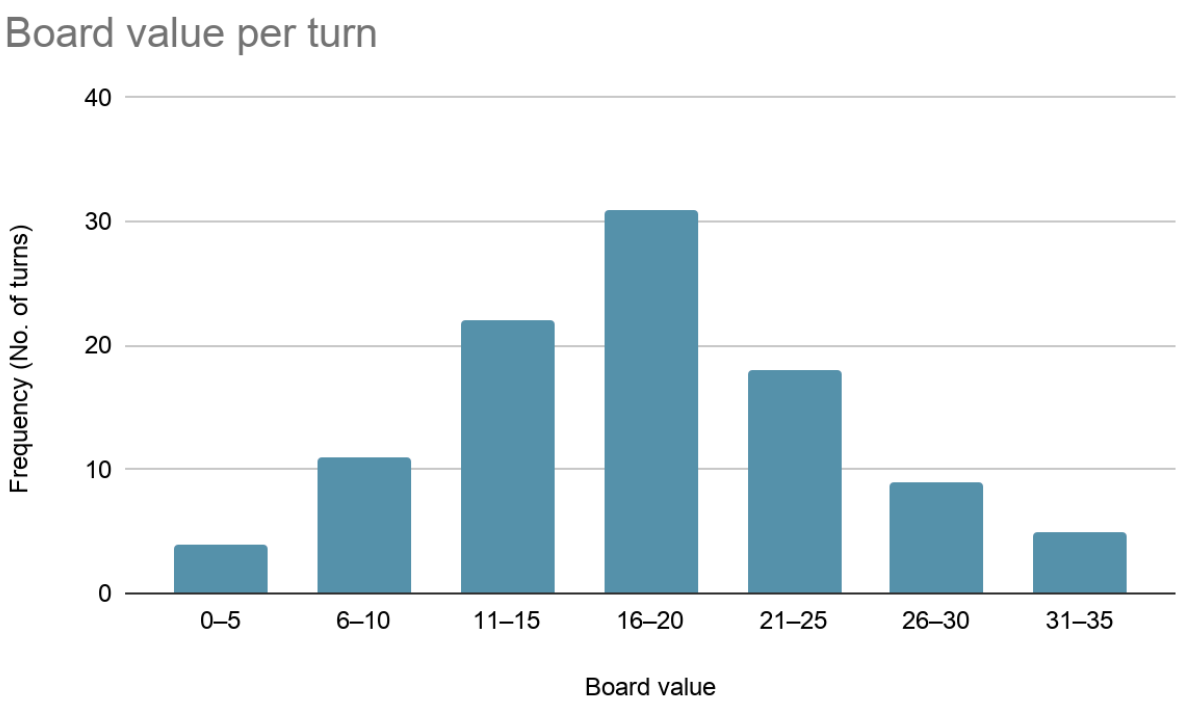
Graph 1: Helper card usage frequency



Graph 2: Board card count (Player vs Demon)



Graph 3: Board value per turn



5. Project Planning Timeline

Week	Week	Task
1	8	Proposal submission / Project initiation
2	9	Full proposal submission
3	10	Finish core gameloop
4	11	Design Demons and Helper Cards
5	12	Implement data recording and analysis
6	13	Polish + debugging game
7	14	Submission week (Draft)

6. Document version

Version: 1.0
Date: 13 March 2025
Editor: Siraphob Phonphakdee

Date	Name	Description of Revision, Feedback, Comments
14/3	Nattanan	- Fix the formatting, using bullet point or bold to help specify the important information - Fix some word and clarify ai enemy (just fix this and you will pass!) - Adding some picture (might be the reference game or you draw the sample) would be nice to visualize your work - The game itself sounds pretty complicated, remember, make sure the player can understand how to play in a few minutes. - Overall good job 👍 nice explanation
21/03	Amornrit	Nice lore. I like it. ● Use bullets to improve readability. ● You might need tutorials for beginners. ● Need more explanation on the AI you mentioned For information, rule-based logic and algorithms are also considered AI. (Good Old-Fashioned AI) It is absolutely fine to use this term, but the details are needed.